single application instance. They're usually categorized according to the type or severity of the event, such as *trace*, *debug*, *info*, *warn*, and *error*. It's a flexible mechanism that lets us log only the most severe events in production while still giving us the chance to change the log level temporarily during debugging.

The format of a log record can vary, going from simple plain text to a more organized collection of key/value pairs to fully structured records produced in a JSON format.

Traditionally we've configured logs to be printed out in files located on the host machine, which has resulted in applications dealing with filename conventions, file rotation, and file sizes. In the cloud we follow the 15-Factor methodology, which recommends treating logs as events streamed to the standard output. Cloud native applications stream logs and are not concerned with how they are processed or stored.

This section will teach you how to add and configure logs in Spring Boot applications. Then I'll explain how logs are collected and aggregated in a cloud native infrastructure. Finally, you'll run Fluent Bit for log collection, run Loki for log aggregation, and use Grafana to query the logs produced by your Spring Boot applications.

13.1.1 Logging with Spring Boot

Spring Boot comes with built-in support and auto-configuration for the most common logging frameworks, including Logback, Log4J2, Commons Logging, and Java Util Logging. By default, Logback is used (https://logback.qos.ch), but you can easily replace it with another library thanks to the abstraction provided by the Simple Logging Facade for Java (SLF4J).